📌 COSA DEVE FARE IL BRIDGE

🔵1️⃣Connettersi via Bluetooth all'Arduino

Dal progetto (Fase acquisizione sensori):

Il Bridge deve:

- aprire la porta seriale Bluetooth (HC-05)
- leggere i messaggi inviati dall'Arduino
- fare parsing del valore gas

Nel file **bridge.py** questo avviene tramite la libreria: `pyserial`

Il Bridge legge stringhe nel formato:

```
MQ2:420
```

La funzione `read_line()`:

- legge la riga seriale
- verifica il formato
- estrae il valore numerico

Output generato:

```
gas = 420

timestamp = ...
```

Esempio struttura restituita:

```
{

 "gas": 420,

 "timestamp": "2026-03-09T21:30:00"

}
```

🔵2️⃣ Validare e loggare il dato

Sempre nella fase di acquisizione:

Il Bridge deve:

- controllare che il valore letto sia numerico
- scartare messaggi non validi
- generare un timestamp della lettura

Questo viene gestito nella funzione:

```
read_line()
```

Se il formato del messaggio non è corretto, la funzione restituisce:

```
None
```

Questo evita che dati non validi entrino nel sistema.

Output interno del Bridge:

```
gas = 420

timestamp = ...
```

🔵3️⃣ Leggere i valori degli altri garage

Per simulare una rete di garage distribuiti, il Bridge utilizza un **dataset CSV**.

Il file contiene valori di gas generati da altri garage simulati.

Il Bridge carica il dataset tramite **pandas**.

Esempio file:

```
dataset_low_pollution.csv
```

Il dataset contiene una colonna:

```
gas_value
```

Esempio valori:

```
others_values = [75, 80, 70, 78, 76]
```

Questi valori rappresentano il livello di gas negli altri garage della rete.

🔵4️⃣ Calcolare statistiche dei garage simulati

Una volta caricato il dataset, il Bridge calcola le statistiche dei valori simulati.

Nel codice:

```
others_mean = df['gas_value'].mean()

others_std = df['gas_value'].std()
```

La statistica principale utilizzata è:

```
others_mean
```

che rappresenta il livello medio di gas negli altri garage.

Questa informazione viene salvata nel Bridge:

```
self.read_dataset
```

Esempio:

```
others_mean = 78
```

🔵5️⃣ Calcolare la soglia di riferimento

Calcolo della soglia spostato a Node-red

🔵6️⃣ Comandare la ventola via Bluetooth

Il Bridge controlla la ventola inviando comandi seriali all'Arduino.

I comandi sono:

```
FAN_ON
```

```
FAN_OFF
```

Nel codice:

```
self.bluetooth.send_command("FAN_ON\n")
```

oppure

```
self.bluetooth.send_command("FAN_OFF\n")
```

Il Bridge mantiene anche uno stato interno della ventola:

```
fan_state = True / False
```

Questo valore viene utilizzato anche nella telemetria MQTT.

⚠️ Importante:

La decisione **non viene più calcolata dal Bridge**, ma arriva da **Node-RED**.

🔵7️⃣ Pubblicare telemetria verso Node-RED (MQTT)

Il Bridge pubblica periodicamente i dati del sistema tramite MQTT.

Topic utilizzato:

```
garages/G1/telemetry
```

Payload JSON inviato:

```
{
  "garage_id": "G1",
  "gas": 420,
  "timestamp": "...",
  "fan_state": true
}
```

Significato dei campi:

campo	descrizione
garage_id	identificatore del garage
gas	valore sensore MQ-2

timestamp	timestamp della lettura
fan_state	stato attuale della ventola

Node-RED utilizza questi dati per:

- visualizzare la dashboard
- calcolare anomalie
- eseguire predizione del gas

🔵8️⃣ Pubblicare eventi di allarme

Il Bridge può pubblicare eventi di sistema sul topic:

```
garages/G1/alerts
```

Attualmente gli alert vengono generati principalmente quando viene eseguito un **override manuale della ventola**.

Esempio payload:

```
{
  "garage_id": "G1",
  "event": "manual_override",
  "command": "FAN_ON",
  "timestamp": "..."
}
```

Node-RED può utilizzare questi eventi per:

- notifiche
- log di sistema
- visualizzazione su dashboard

🔵9️⃣ Ricevere comandi da Node-RED

Il Bridge riceve comandi tramite MQTT.

Topic:

```
garages/G1/cmd
```

Node-RED invia due tipi di comandi.

Comandi automatici

Node-RED esegue:

- analisi dei dati
- predizione del gas
- decisione operativa

e invia al Bridge un payload JSON.

Esempio:

```
{
 "mode": "STD",
 "anomaly": true,
 "predicted_crossing": false
}
```

Il Bridge applica la logica:

```
se anomaly == True
oppure predicted_crossing == True
```

allora:

```
FAN_ON
```

altrimenti:

```
FAN_OFF
```

Comandi manuali (override)

Node-RED può anche forzare manualmente lo stato della ventola. Comandi supportati:

```
FAN_ON
FAN_OFF
AUTO
```

Significato:

comando	effetto
FAN_ON	accensione forzata
FAN_OFF	spegnimento forzato
AUTO	ritorno alla modalità automatica

Quando è attivo **manual_override**, il Bridge ignora i comandi automatici.

🎯 RIASSUNTO FINALE – COSA CONTIENE bridge.py

Il Bridge implementa i seguenti moduli logici:

1️⃣ Connessione Bluetooth con Arduino (pyserial)
2️⃣ Lettura e parsing del valore MQ-2
5️⃣ Controllo della ventola tramite Bluetooth
6️⃣ Pubblicazione telemetria MQTT
7️⃣ Pubblicazione eventi di sistema MQTT
8️⃣ Ricezione comandi automatici da Node-RED
9️⃣ Supporto override manuale della ventola

Il Bridge funge quindi da **gateway IoT tra hardware e infrastruttura di controllo**.

🔷 "Flusso completo del sistema OffGas"

con schema tipo:

Arduino → Bridge → MQTT Broker → Node-RED

↓

decisione

↓

Arduino ← Bridge ← MQTT Broker ← Node-RED

📁 Bridge IoT – OffGas

Documento operativo e descrittivo che combina:
(1) descrizione architetturale del Bridge implementato a classi
(2) roadmap a task step-by-step per costruirlo.

Panoramica del Bridge OffGas

Il Bridge è il componente centrale del sistema OffGas. Il suo compito è collegare tre elementi principali del sistema IoT:

- Arduino + sensore MQ-2 + ventola tramite comunicazione Bluetooth (modulo HC-05)
- Dataset CSV che simula i valori di gas degli altri garage
- Broker MQTT e Node-RED, utilizzati per la telemetria e per l'invio di comandi manuali

L'obiettivo del sistema è implementare un controllo distribuito e comparativo della qualità dell'aria. Il valore di gas rilevato nel garage locale (G1) viene confrontato con la media dei valori degli altri garage. Se il valore locale supera una soglia relativa, la ventola viene accesa; altrimenti viene spenta.

La soglia è definita dalla formula:

```
gas_locale > media_altri_garage * k
```

Configurazione del sistema (Config)

La classe `Config` contiene tutti i parametri configurabili del sistema.

```
class Config:
     SERIAL_PORT = "COM6"
     BAUD_RATE = 9600

     MQTT_BROKER = "localhost"
     MQTT_PORT = 1883
```

```
     TOPIC_TELEMETRY = "garages/G1/telemetry"  # G1 è il
garage locale
     TOPIC_ALERTS = "garages/G1/alerts"
     TOPIC_COMMANDS = "garages/G1/cmd"

     ANOMALY_FACTOR = 1.5
     COOLDOWN_SECONDS = 5
```

Nota sui topic: 'G1' identifica il garage locale; nel dataset sono presenti G2, G3, ... come garage simulati.

Architettura a classi (bridge.py)

Il Bridge è strutturato per responsabilità, con queste classi principali:

BluetoothManager – acquisizione valori MQ-2 e invio comandi **FAN_ON / FAN_OFF** ad Arduino
FanController – gestione stato ventola e supporto **manual override**
MQTTManager – gestione telemetria MQTT e ricezione comandi da Node-RED
Bridge – coordinamento del ciclo principale del sistema

⚠️ Differenza rispetto alla versione iniziale del progetto:

La **logica di decisione dell'anomalia non è più nel Bridge**.

La valutazione della condizione di anomalia e della predizione del gas viene eseguita in **Node-RED**, che invia al Bridge il risultato tramite MQTT.

Il Bridge diventa quindi un **gateway IoT edge**, responsabile di:

- acquisizione dati sensore
- pubblicazione telemetria
- esecuzione comandi di controllo

Roadmap a Task (step-by-step)

I task sotto rappresentano una roadmap incrementale. Ogni task richiama la parte corrispondente del codice già implementato e ne spiega responsabilità, input/output e snippet chiave.

0) Import librerie necessarie

Librerie usate nel Bridge:

- serial → comunicazione Bluetooth (pyserial)

- json → formattazione messaggi MQTT
- time → loop e throttling
- datetime → timestamp
- pandas → lettura dataset CSV e statistiche
- paho.mqtt.client → comunicazione MQTT

Altre import presenti (sys, os, threading, statistics) sono disponibili per estensioni / future evoluzioni.

Installazione: `pip install pyserial paho-mqtt pandas`

1) TASK: Connessione Bluetooth
Obiettivo: aprire la porta seriale del modulo HC-05 e iniziare la comunicazione con Arduino.

Classe: BluetoothManager

Snippet:

```
def connect(self):
    self.ser = serial.Serial(
     port=self.port,
        baudrate=self.baud_rate,
     timeout=self.timeout
    )
```

Nota: in `Bridge.start()` viene chiamato `self.bluetooth.connect()` prima del loop.

2) TASK: Lettura e parsing del dato gas
Obiettivo: leggere una riga dalla seriale, decodificare bytes→string, validare formato, estrarre valore, aggiungere timestamp.

Formato atteso: `MQ2:<valore>`

```
def read_line(self):
    read_val = self.ser.readline().decode('utf-8').strip()

     if read_val.startswith("MQ2:"):
        val_gas = int(read_val.split(':',1)[1])
     timestamp = datetime.datetime.now().isoformat()
     return {"gas": val_gas, "timestamp": timestamp}
```

Output: dizionario con `gas` e `timestamp` (o `None` se parsing fallisce).

3)4)calcolo della soglia su Node-Red

5) TASK: Logica di controllo esterna (Node-RED)

Nella versione attuale del sistema OffGas la logica di rilevamento anomalia non viene più eseguita nel Bridge.

Il Bridge **non calcola più direttamente la condizione**

```
gas_locale > media_altri_garage * k
```

Questa logica viene ora implementata in **Node-RED**, tramite nodi Function.

Node-RED utilizza:

- valore gas inviato dal Bridge
- soglia calcolata dal Bridge
- predizione temporale del gas

per decidere se attivare la ventilazione.

Il risultato viene inviato al Bridge sotto forma di comando JSON.

Esempio payload ricevuto dal Bridge:

```
{
 "mode": "STD",
 "anomaly": true,
 "predicted_crossing": false
}
```

campo	significato
mode	modalità operativa

anomaly	valore attuale sopra soglia
predicted_crossing	previsione superamento soglia

6) TASK: Gestione stato ventola

La gestione della ventola è implementata nella classe **FanController**.

```
Questa classe mantiene lo stato corrente della ventola:

self.fan_state = False
```

dove:

```
False → ventola spenta

True  → ventola accesa
```

Il controllo avviene tramite i metodi:

```
def force_on(self):

   self.fan_state = True

   self.bluetooth.send_command("FAN_ON\n")

def force_off(self):

   self.fan_state = False

   self.bluetooth.send_command("FAN_OFF\n")
```

I comandi inviati ad Arduino sono quindi:

```
FAN_ON

def force_on(self):
    self.fan_state = True
    self.bluetooth.send_command("FAN_ON\n")

FAN_OFF

def force_off(self):
    self.fan_state = False
    self.bluetooth.send_command("FAN_OFF\n")
```

La scrittura sulla seriale è gestita da **BluetoothManager**:

```
def send_command(self, command: str):
   self.ser.write(command.encode("utf-8"))
```

7) TASK: Ricezione decisione da Node-RED

Il Bridge riceve la decisione operativa tramite MQTT.

```
Topic:
garages/G1/cmd
```

La gestione dei messaggi avviene nella funzione:

```
def on_message(self, client, userdata, msg):
```

Il payload MQTT viene decodificato:

```
raw = msg.payload.decode("utf-8").strip()
command = json.loads(raw)
```

Il Bridge utilizza la seguente logica di controllo:

```
if command["mode"] == "STD":
    if command["anomaly"] or command["predicted_crossing"]:
        self.fan_controller.force_on()
    else:
        self.fan_controller.force_off()
```

Quindi la ventola viene attivata quando:

```
anomaly == True
```

oppure quando:

```
predicted_crossing == True
```

Questo consente al sistema di:

- reagire a condizioni di gas già sopra soglia
- anticipare situazioni di rischio tramite predizione

---

8) TASK: Connessione MQTT

MQTT si basa su tre componenti principali:

componente	ruolo
Publisher	dispositivo che invia messaggi
Subscriber	dispositivo che riceve messaggi
Broker	server centrale che gestisce lo scambio

Nel sistema OffGas:

componente	ruolo
Bridge	publisher + subscriber
Node-RED	publisher + subscriber
MQTT Broker	server di comunicazione

Il broker funge da **intermediario centrale** tra tutti i dispositivi.

Il **broker MQTT** è un server che:

- riceve i messaggi pubblicati dai client
- li distribuisce ai client interessati

- gestisce le sottoscrizioni ai topic
- mantiene le connessioni dei dispositivi

Nel progetto OffGas il broker utilizzato è configurato in:

```
MQTT_BROKER = "localhost"

MQTT_PORT = 1883
```

MQTT utilizza un modello **publish/subscribe**, diverso dal classico modello client-server.

In questo modello:

- un dispositivo **pubblica** un messaggio su un **topic**
- il broker riceve il messaggio
- il broker lo inoltra a tutti i dispositivi **sottoscritti a quel topic**

Esempio nel sistema OffGas:

```
Bridge → publish → garages/G1/telemetry
```

Node-RED è sottoscritto a quel topic e riceve automaticamente i dati.

Allo stesso modo:

```
Node-RED → publish → garages/G1/cmd
```

Il Bridge è sottoscritto a questo topic e riceve i comandi.

Nel Bridge sono definiti tre topic principali:

```
TOPIC_TELEMETRY = "garages/G1/telemetry"

TOPIC_ALERTS = "garages/G1/alerts"

TOPIC_COMMANDS = "garages/G1/cmd"
```

topic	funzione
garages/G1/telemetry	telemetria sensori
garages/G1/alerts	eventi e notifiche
garages/G1/cmd	comandi da Node-RED

La comunicazione con il broker MQTT è gestita dalla classe **MQTTManager**.

Il client MQTT viene creato tramite la libreria **paho-mqtt**.

```
self.client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
```

Questo oggetto rappresenta il **client MQTT** che:

- stabilisce la connessione al broker
- invia messaggi
- riceve messaggi
- gestisce le sottoscrizioni ai topic

MQTT è un protocollo **event-driven**.

Questo significa che il client reagisce agli eventi tramite **callback functions**.

La callback per i messaggi è registrata con:

```
self.client.on_message = self.on_message
```

Questo indica che ogni volta che il client riceve un messaggio su un topic sottoscritto, verrà eseguita la funzione:

```
def on_message(self, client, userdata, msg)
```

campo	significato
msg.topic	topic del messaggio
msg.payload	contenuto del messaggio

Connessione al broker:

```
def connect(self):
    self.client.connect(self.broker, self.port)
    self.client.loop_start()
```

---

9) TASK: Pubblicazione telemetria

Il Bridge pubblica periodicamente lo stato del sistema su MQTT.

```
Topic:

garages/G1/telemetry
```

Payload generato nel metodo:

```
process_cycle(self, data)
```

Esempio payload:

```
payload = {
 "garage_id": "G1",
 "gas": data["gas"],
 "threshold": self.read_dataset['others_mean'] *
self.config.ANOMALY_FACTOR,
 "timestamp": data["timestamp"],
 "fan_state": self.fan_controller.fan_state
```

```
}
```

Questo messaggio viene inviato tramite:

```
self.mqtt.publish_telemetry(payload)
```

La telemetria viene utilizzata da Node-RED per:

- visualizzazione dashboard
- analisi dati
- predizione del gas

---

10) TASK: Pubblicazione alert

Gli alert vengono pubblicati quando avvengono eventi rilevanti nel sistema.

Attualmente vengono generati principalmente durante **manual override della ventola**.

Topic MQTT:

```
garages/G1/alerts
```

Esempio payload:

```
{
 "garage_id": "G1",
 "event": "manual_override",
 "command": "FAN_ON",
 "timestamp": "..."
}
```

La pubblicazione avviene tramite:

```
self.publish_alert(payload)
```

---

11) TASK: Comandi manuali da Node-RED

Node-RED può forzare manualmente lo stato della ventola.

Comandi supportati:

```
FAN_ON
FAN_OFF
AUTO
```

La gestione avviene nella funzione **on_message**.

Esempio:

```
if command in ("FAN_ON", "ON"):
    self.fan_controller.manual_override = True
    self.fan_controller.force_on()

elif command in ("FAN_OFF", "OFF"):
    self.fan_controller.manual_override = True
    self.fan_controller.force_off()

elif command in ("AUTO_MODE", "AUTO"):
    self.fan_controller.set_auto_mode()
```

Significato:

comando	effetto
FAN_ON	forza accensione
FAN_OFF	forza spegnimento
AUTO	ritorno alla modalità automatica

Quando `manual_override = True`, il Bridge ignora i comandi automatici provenienti da Node-RED.

Bridge – Coordinamento del ciclo runtime

Il Bridge esegue un ciclo continuo di acquisizione e pubblicazione dati.

Loop principale:

```
while self.running:

    data = self.bluetooth.read_line()

    if data:
        self.process_cycle(data)

    time.sleep(0.2)
```

Il metodo `process_cycle()` pubblica la telemetria MQTT.

```
payload = {
 "garage_id": "G1",
 "gas": data["gas"],
 "threshold": self.read_dataset['others_mean'] *
self.config.ANOMALY_FACTOR,
 "timestamp": data["timestamp"],
 "fan_state": self.fan_controller.fan_state
}

self.mqtt.publish_telemetry(payload)
```

Rimozione del modulo di predizione dal Bridge

Nella versione iniziale del progetto era prevista una classe `PredictionEngine` nel Bridge.

Nel sistema attuale questa logica è stata **spostata in Node-RED**, che implementa:

- moving average
- calcolo del trend
- predizione del gas
- valutazione `predicted_crossing`

Questo approccio permette di:

- semplificare il Bridge
- rendere modificabile la logica senza cambiare il codice Python
- sfruttare le capacità di elaborazione di Node-RED.

Checklist finale

Il Bridge OffGas implementa:

✔ Lettura gas da Arduino via Bluetooth (`MQ2:<valore>`)
✔ Parsing del valore MQ-2
✔ Caricamento dataset garage simulati
✔ Calcolo soglia di riferimento
✔ Pubblicazione telemetria MQTT (`garages/G1/telemetry`)
✔ Ricezione decisioni automatiche da Node-RED
✔ Controllo ventola tramite Arduino
✔ Supporto manual override (`FAN_ON`, `FAN_OFF`, `AUTO`)
✔ Pubblicazione alert MQTT (`garages/G1/alerts`)